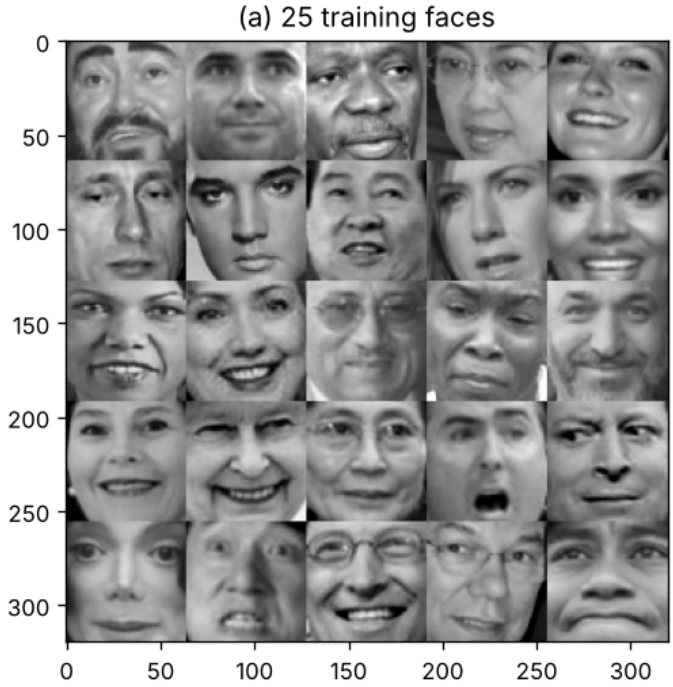
Problem 1

(a)

Each 64 × 64 image is read with `imread` and flattened into a column vector $\mathbf{f}_i$ of length 4096. The 25 columns are stored in the matrix $A = [\mathbf{f}_1\ \mathbf{f}_2\ \dots\ \mathbf{f}_{25}]$ of size 4096 × 25 (`trainingFaces` in the code).

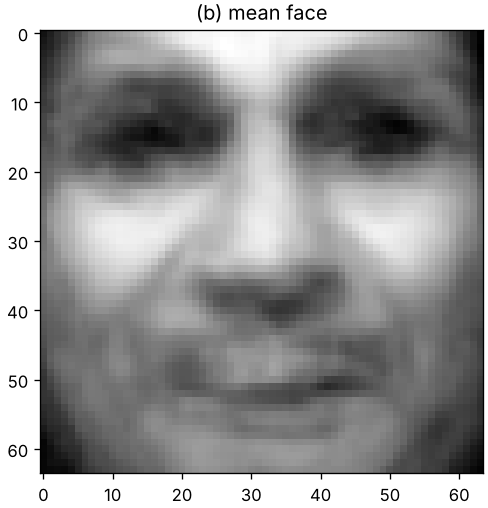
```
pgm_files = glob('./facedata/training/*.pgm')
M = len(pgm_files)
img = io.imread(pgm_files[0], as_gray=True); m, n = img.shape
trainingFaces = np.zeros((m*n, M))
for i in range(M):
    img = io.imread(pgm_files[i], as_gray=True)
    trainingFaces[:, i] = img.flatten()
```



(b)

The mean image is the average of the 25 columns of A, i.e. the pixel-wise average of the training faces.

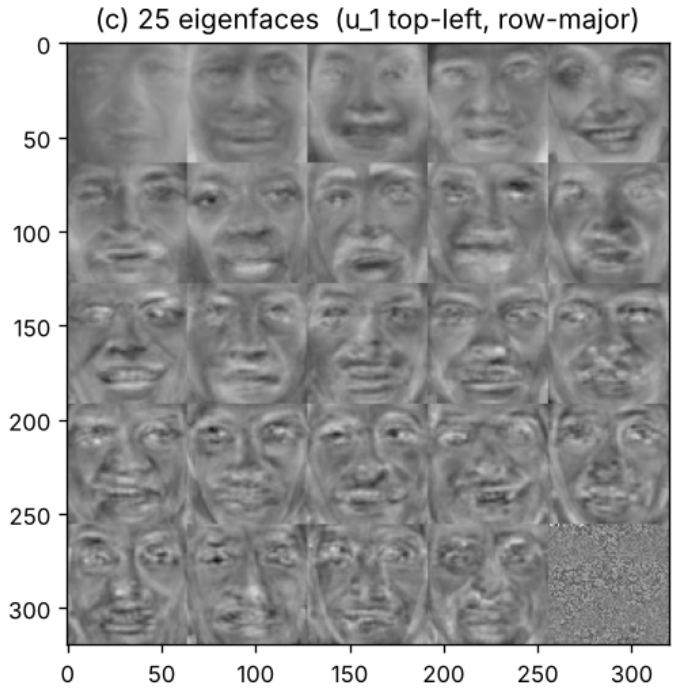
```
avgFace = np.mean(trainingFaces, axis=1)[:, np.newaxis]
```



(c)

The mean face is subtracted from every column, $X = A - \bar{\mathbf{f}}\,1^T$, and the SVD $X = U\,S\,V^T$ is computed. The columns $\mathbf{u}_k$ of U (4096 × 25) are the eigenfaces.

```
X = trainingFaces - np.tile(avgFace, (1, M))
U, S, VT = np.linalg.svd(X, full_matrices=False)
img = U[:, i]     # i-th eigenface, reshaped to 64 x 64 for plotting
```



Unlike the original images, the eigenfaces are not pictures of any particular person. They are orthonormal directions in pixel space along which the 25 mean-subtracted faces vary the most: $\mathbf{u}_1$ captures the largest variation (overall shading and face outline), and later eigenfaces

capture finer, noisier details with smaller singular values. Their entries are positive and negative with unit norm, so they appear with a very different contrast from the original 0–255 images. The last singular value is 0 (the 25 centered columns sum to zero, so rank X = 24), so $\mathbf{u}_{25}$ is an arbitrary vector orthogonal to the data and looks like noise.

(d)

Since the $\mathbf{u}_k$ are orthonormal, the coefficients of face i in the eigenface basis are $c_{ki} = \mathbf{u}_k^T(\mathbf{f}_i - \bar{\mathbf{f}})$. For all faces at once this is the 25 × 25 matrix $W = U^T X$, whose i-th column holds the coordinates of face i. The reconstruction $U W + \bar{\mathbf{f}}\, 1^T$ recovers A exactly.

```
W = U.T @ X     # column i = coordinates of face i in the eigenface basis
```

Advantage: in the original basis each image needs 4096 numbers; in the eigenface basis it needs only 25 (in fact 24 nonzero) numbers, with no loss of information for the training set. Comparing faces (distances) in a 25-dimensional space is also far cheaper than in 4096 dimensions, and dropping the eigenfaces with small singular values gives a compact low-rank approximation.

(e)

The input face is centered and projected onto the eigenfaces, $\mathbf{w} = U^T(\mathbf{f} - \bar{\mathbf{f}})$, and the Euclidean distance between **w** and each column of W is computed. Jackie Chan was used as the input.

```
test = io.imread('./facedata/training/Jackie_Chan_0006.pgm', as_gray=True)
f = test.flatten()[:, np.newaxis]
w = U.T @ (f - avgFace)
dist = np.linalg.norm(W - w, axis=0)
best = np.argmin(dist)
```

```
      0.00  Jackie_Chan_0006.pgm
   2419.54  Pervez_Musharraf_0001.pgm
   2752.09  Vladimir_Putin_0026.pgm
   2833.50  Hamid_Karzai_0011.pgm
   ...
```

The best match is Jackie_Chan_0006.pgm with distance 0, because the input is one of the training images, so its coordinates are exactly a column of W. The next closest face is at distance 2419. The algorithm works.

(f)

```
clint = io.imread('./facedata/test/Clint_Eastwood_0005.pgm', as_gray=True)
f = clint.flatten()[:, np.newaxis]
w = U.T @ (f - avgFace)
dist = np.linalg.norm(W - w, axis=0)
best = np.argmin(dist)
recon = U @ w + avgFace      # eigenface representation
```

```
   1894.08  Hamid_Karzai_0011.pgm
   1938.02  Bill_Gates_0010.pgm
```

```
2280.33  Yoko_Ono_0001.pgm
2327.91  Pervez_Musharraf_0001.pgm
...
```

The closest match for Clint Eastwood is Hamid Karzai (distance 1894), with Bill Gates a close second (1938).



The input image and its eigenface representation are not the same. Clint Eastwood is not in the training set, so $\mathbf{f} - \bar{\mathbf{f}}$ does not lie in the 24-dimensional subspace spanned by the eigenfaces. $U\mathbf{w} + \bar{\mathbf{f}}$ is only the orthogonal projection of the image onto that subspace; the component orthogonal to it is lost. Numerically, $||\mathbf{f} - \text{recon}|| = 1709$ while $||\mathbf{f} - \bar{\mathbf{f}}|| = 2354$, so a substantial part of what distinguishes Clint Eastwood from the mean face is discarded. The reconstruction keeps the overall face shape and expression and is still recognizable, but it is smoother and loses fine detail. In contrast, in (e) the input was in the training set, so its projection was exact.

To make the eigenface representation more accurate, the subspace must be enlarged: use many more training images (more people, and several photos of each person under different poses and lighting) so that the eigenfaces span a richer space of facial variation. Including other photos of Clint Eastwood in the training set would improve both the reconstruction and the recognition. With only 25 images, only 24 directions are available, which is far too few to represent an unseen face.

Problem 2

(a)

Each of the M = 302 frames (500 × 500, grayscale) is flattened into a column of length 250000, giving a 250000 × 302 matrix. The mean frame is subtracted from every column and the reduced SVD $X = U S V^T$ is computed with `full_matrices=0`, so U is 250000 × 302 rather than 250000 × 250000.

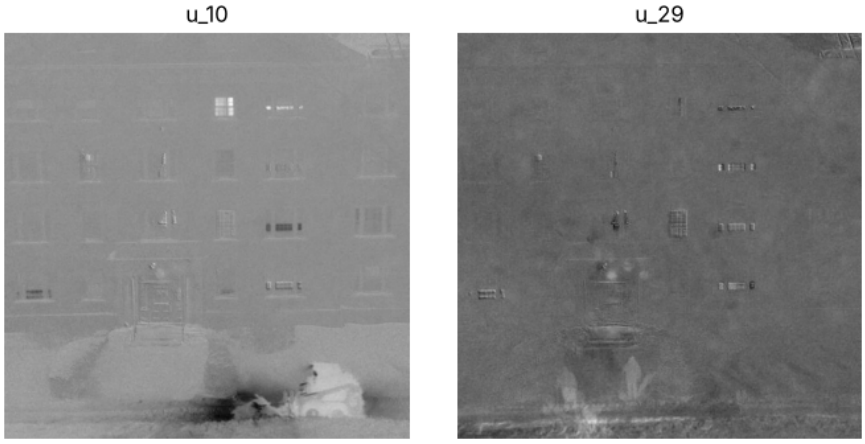
```
trainingDorms[:, i] = img.flatten()
avgDorm = np.mean(trainingDorms, axis=1)[:, np.newaxis]
img = np.reshape(avgDorm, (m, n))
X = trainingDorms - np.tile(avgDorm, (1, M))
U, S, VT = np.linalg.svd(X, full_matrices=0)
```

The first four left singular vectors, reshaped to 500 × 500:



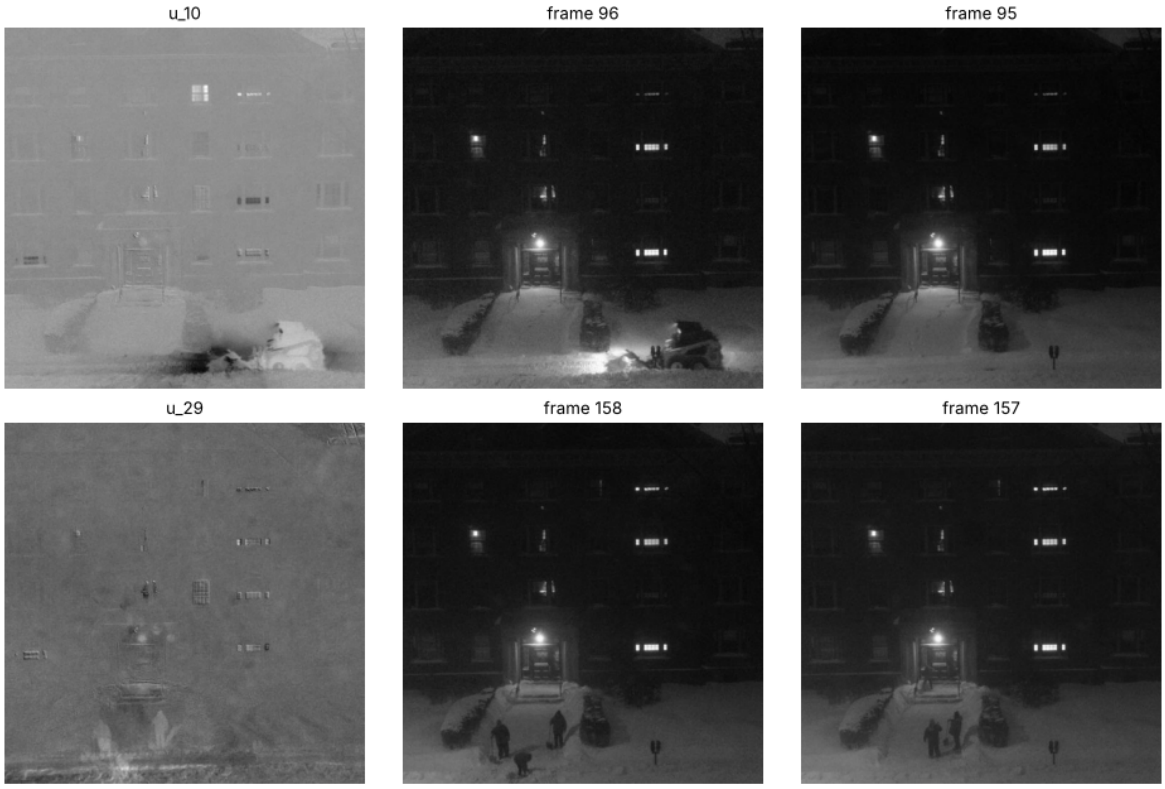
$\mathbf{u}_1$ is the outline of the whole building with the lit windows and entrance; $\mathbf{u}_2$ isolates one very bright window; $\mathbf{u}_3$ is an overall brightness pattern of the facade and the snow-covered ground; $\mathbf{u}_4$ is a single column of windows on the right.

(b)

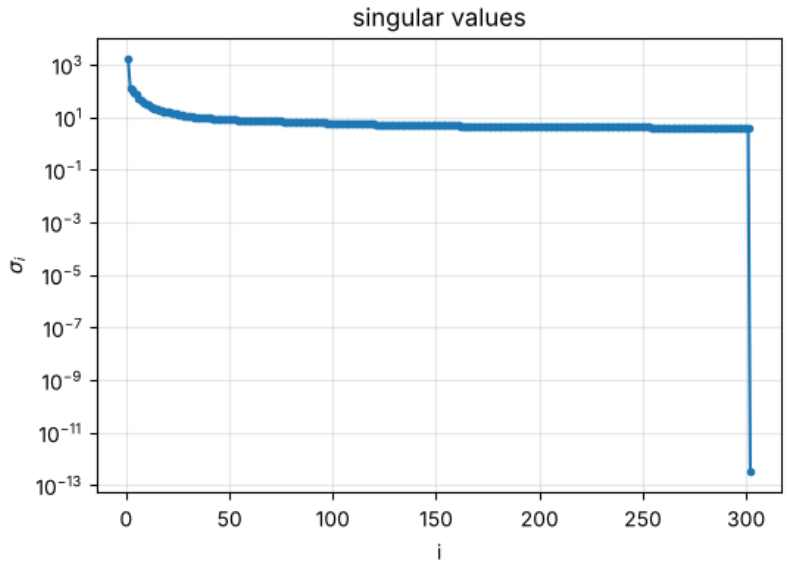


Both contain transient events rather than lighting patterns. $\mathbf{u}_{10}$ shows a bright object in the lower right corner: its time coefficient $v_{10}(t)$ is dominated by a single frame (frame 96, coefficient −0.87), and that frame shows a snow plow driving past the entrance. $\mathbf{u}_{29}$ shows

several human figures in front of the entrance; its largest coefficient is at frame 158 (–0.38), followed by frames 18 and 159; frames 158–159 show people shoveling snow in front of the entrance, which is where the figures come from. The SVD puts events that appear in only a few frames into later singular vectors, since they explain only a small share of the total variance.

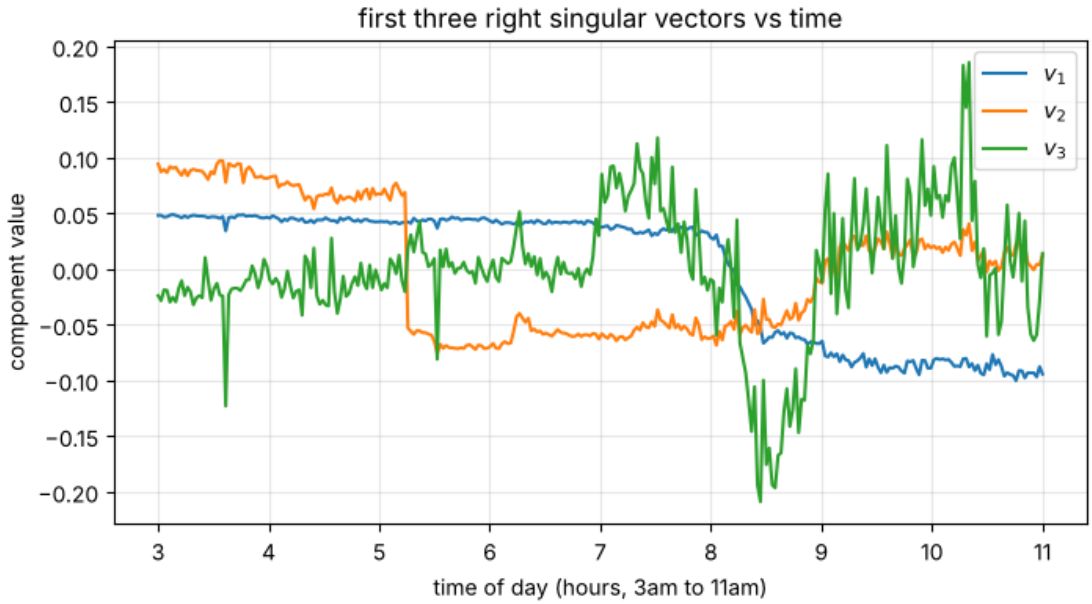


(c)



σ_1 = 1638.6 is about 12 times larger than σ_2 = 132.7; after that the singular values decay slowly into a long tail of single-digit values, and the last one is zero because the mean-subtracted columns sum to zero (rank X = 301). The cumulative energy $\Sigma_{i\leq k}\sigma_i^2 / \Sigma\sigma_i^2$ is 0.976 for k = 1, 0.992 for k = 5, 0.995 for k = 10 and 0.996 for k = 20. Given the sharp drop after the first singular value and the flattening after about 10, keeping roughly 5 to 10 singular vectors is a reasonable low-rank approximation for compression; going beyond ~20 buys very little.

(d)



The 302 frames span 3am to 11am, so each $v_k(t)$ is the weight of the pattern $\mathbf{u}_k$ at time t. $\mathbf{v}_1$ is nearly constant and positive until about 8am and then switches to negative between 8 and

9am: this is sunrise, the single largest change in the movie, when the whole scene goes from dark with lit windows to daylight. $\mathbf{v}_2$ has a sharp step at about 5:15am and a second change near 8:30am; combined with $\mathbf{u}_2$, which isolates one bright window, the step corresponds to that window's light being switched off. $\mathbf{v}_3$ is small and noisy at night and fluctuates strongly between 7 and 9am, the period with the most activity in the scene (people shoveling, changing snowfall and illumination).

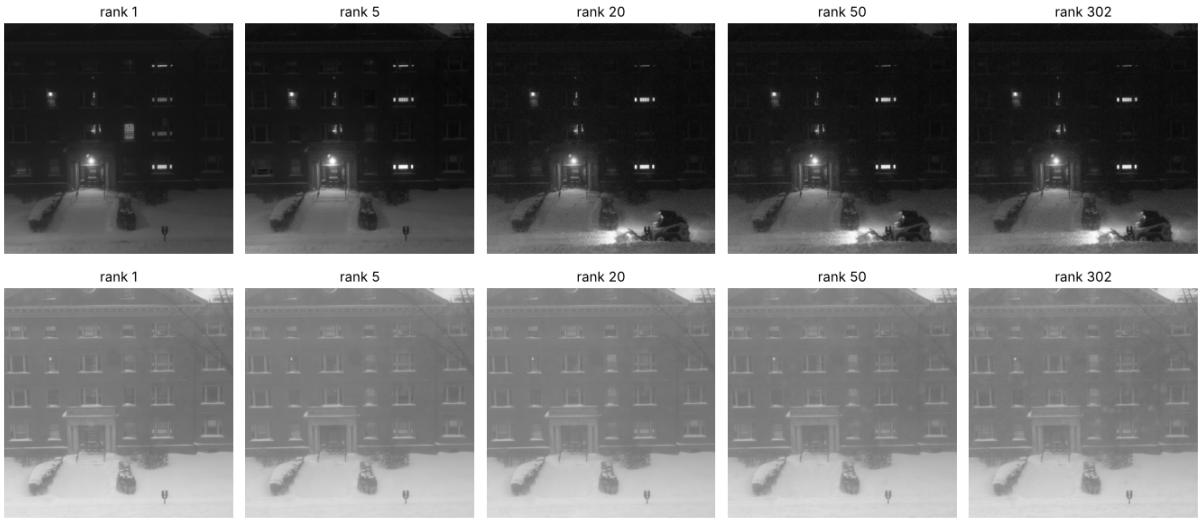
(e)

The rank-k approximation of the movie is $X_k = U_k S_k V_k^T$ (first k columns of U, first k singular values, first k rows of V^T), and the frames are the columns of X_k plus the mean frame. `play(k)` displays the rank-k movie for any k from 1 to M.

```
def lowrank_movie(k):
    Xk = U[:, :k] @ (S[:k, np.newaxis] * VT[:k, :])
    return Xk + avgDorm                      # pixels x M, column t = frame t

def play(k):
    frames = lowrank_movie(k)
    ax = None
    for i in range(M):
        img = np.reshape(frames[:, i], (m, n))
        if ax is None:
            ax = plt.imshow(img, cmap=plt.cm.gray, vmin=0, vmax=1)
        else:
            ax.set_data(img)
        plt.pause(.001); plt.draw()
```

Frame 96 (night, snow plow) and frame 250 (daylight) at ranks 1, 5, 20, 50 and 302:



Rank 1 reproduces only the mean building plus the day/night change; the snow plow is missing. It appears at rank 20 because $\mathbf{u}_{10}$ is then included. Ranks 50 and 302 are visually indistinguishable. The relative Frobenius error $||X - X_k|| / ||X||$ is 0.155 (k = 1), 0.091 (k = 5), 0.072 (k = 10) and 0.060 (k = 20), while a rank-10 movie needs only 3.3% of the original storage.